

PostgreSQL Administration – Lab Exercises

Lab prerequisites

- Ubuntu Linux server
- PostgreSQL 18 installed
- Administrative access through `sudo`
- Default PostgreSQL cluster running on port `5432`
- PostgreSQL commands executed using the `postgres` operating-system account

Verify the environment:

Bash

```
psql --version
pg_lsclusters
sudo systemctl status postgresql
```

Lab 1 — Database Creation and Management

Objective

Create, inspect, modify, connect to, and remove a PostgreSQL database.

Exercise 1.1 — Connect as administrator

Bash

```
sudo -iu postgres psql
```

Check the current connection:

SQL

```
SELECT current_database(), current_user;
```

Exercise 1.2 — Create the database owner

SQL

```
CREATE ROLE banking_owner
LOGIN
PASSWORD 'BankingOwner@123';
```

Use a secure secret-management method rather than a password embedded in SQL in production.

Exercise 1.3 — Create a database

SQL

```
CREATE DATABASE banking_db
  OWNER banking_owner
  ENCODING 'UTF8'
  TEMPLATE template0;
```

List databases:

SQL

```
\l
```

Inspect the database:

SQL

```
SELECT datname,
       pg_get_userbyid(datdba) AS owner,
       pg_encoding_to_char(encoding) AS encoding
FROM pg_database
WHERE datname = 'banking_db';
```

Exercise 1.4 — Connect to the database

SQL

```
\c banking_db
```

Verify:

SQL

```
SELECT current_database(), current_user;
```

Exercise 1.5 — Create sample objects

SQL

```
CREATE SCHEMA banking AUTHORIZATION banking_owner;
```

```
CREATE TABLE banking.accounts (  
    account_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    customer_name VARCHAR(100) NOT NULL,  
    balance NUMERIC(12,2) NOT NULL CHECK (balance >= 0),  
    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP  
);
```

Insert and query data:

SQL

```
INSERT INTO banking.accounts (customer_name, balance)  
VALUES  
    ('Anita', 50000),  
    ('Bala', 35000);  
  
SELECT * FROM banking.accounts;
```

Exercise 1.6 — Rename the database

First connect to another database:

SQL

```
\c postgres
```

Rename it:

SQL

```
ALTER DATABASE banking_db  
RENAME TO banking_training_db;
```

Verify:

SQL

```
\1
```

Exercise 1.7 — Configure database connection limit

SQL

```
ALTER DATABASE banking_training_db  
CONNECTION LIMIT 20;
```

Verify:

SQL

```
SELECT datname, datconnlimit
FROM pg_database
WHERE datname = 'banking_training_db';
```

Cleanup

SQL

```
DROP DATABASE banking_training_db;
DROP ROLE banking_owner;
```

Lab 2 — PostgreSQL Cluster Management

Objective

Create and administer an additional PostgreSQL cluster without changing the default cluster.

In PostgreSQL, a cluster is one server instance containing databases, roles, configuration files, WAL files, and data files.

Exercise 2.1 — Examine existing clusters

Exit `psql`:

SQL

```
\q
```

List clusters:

Bash

```
pg_lsclusters
```

Expected example:

Ver	Cluster	Port	Status	Owner	Data directory
18	main	5432	online	postgres	/var/lib/postgresql/18/main

Exercise 2.2 — Create a training cluster

Confirm port `5433` is available:

Bash

```
sudo ss -ltnp | grep ':5433'
```

No output means that nothing is listening on that port.

Create and start the cluster:

Bash

```
sudo pg_createcluster 18 training --port 5433 --start
```

Verify:

Bash

```
pg_lsclusters
```

Exercise 2.3 — Connect to the new cluster

Bash

```
sudo -iu postgres psql -p 5433 -d postgres
```

Inspect its configuration:

SQL

```
SELECT version();

SHOW port;
SHOW data_directory;
SHOW config_file;
SHOW hba_file;
```

Exit:

SQL

```
\q
```

Exercise 2.4 — Start, stop and restart the cluster

Check status:

Bash

```
sudo pg_ctlcluster 18 training status
```

Stop:

Bash

```
sudo pg_ctlcluster 18 training stop  
pg_lsclusters
```

Start:

Bash

```
sudo pg_ctlcluster 18 training start  
pg_lsclusters
```

Restart:

Bash

```
sudo pg_ctlcluster 18 training restart
```

Exercise 2.5 — Reload configuration

Find the configuration file:

Bash

```
sudo -iu postgres psql -p 5433 -d postgres \  
-c "SHOW config_file;"
```

Edit it:

Bash

```
sudo nano /etc/postgresql/18/training/postgresql.conf
```

Add or update:

conf

```
log_connections = on
```

Reload without disconnecting users:

Bash

```
sudo pg_ctlcluster 18 training reload
```

Validate:

Bash

```
sudo -iu postgres psql -p 5433 -d postgres \  
-c "SHOW log_connections;"
```

Exercise 2.6 — Create a database in the training cluster

Bash

```
sudo -iu postgres createdb -p 5433 cluster_lab_db
```

Verify:

Bash

```
sudo -iu postgres psql -p 5433 -d postgres -c "\l"
```

Cleanup

The following command permanently deletes the training cluster and all databases inside it:

Bash

```
sudo pg_dropcluster 18 training --stop
```

Verify:

Bash

```
pg_lsclusters
```

Lab 3 — Roles and Privilege Management

Objective

Create group roles and implement read-only and read-write access.

Exercise 3.1 — Create the training database

Bash

```
sudo -iu postgres psql
```

SQL

```
CREATE DATABASE role_lab_db;  
\c role_lab_db
```

Exercise 3.2 — Create group roles

Group roles usually do not have login permission:

SQL

```
CREATE ROLE banking_readonly NOLOGIN;  
CREATE ROLE banking_readwrite NOLOGIN;
```

Verify:

SQL

```
\du
```

Exercise 3.3 — Create a schema and table

SQL

```
CREATE SCHEMA banking;  
  
CREATE TABLE banking.customers (  
    customer_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    customer_name VARCHAR(100) NOT NULL,  
    city VARCHAR(50),  
    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP  
);  
  
INSERT INTO banking.customers (customer_name, city)  
VALUES  
    ('Arun', 'Chennai'),  
    ('Meena', 'Bengaluru');
```

Exercise 3.4 — Grant read-only permissions

SQL

```
GRANT CONNECT ON DATABASE role_lab_db TO banking_readonly;
GRANT USAGE ON SCHEMA banking TO banking_readonly;

GRANT SELECT
ON ALL TABLES IN SCHEMA banking
TO banking_readonly;
```

Grant access automatically on future tables created by the current owner:

SQL

```
ALTER DEFAULT PRIVILEGES IN SCHEMA banking
GRANT SELECT ON TABLES TO banking_readonly;
```

Exercise 3.5 — Grant read-write permissions

SQL

```
GRANT CONNECT ON DATABASE role_lab_db TO banking_readwrite;
GRANT USAGE ON SCHEMA banking TO banking_readwrite;

GRANT SELECT, INSERT, UPDATE, DELETE
ON ALL TABLES IN SCHEMA banking
TO banking_readwrite;

GRANT USAGE, SELECT
ON ALL SEQUENCES IN SCHEMA banking
TO banking_readwrite;
```

Default privileges for future objects:

SQL

```
ALTER DEFAULT PRIVILEGES IN SCHEMA banking
GRANT SELECT, INSERT, UPDATE, DELETE
ON TABLES TO banking_readwrite;

ALTER DEFAULT PRIVILEGES IN SCHEMA banking
GRANT USAGE, SELECT
ON SEQUENCES TO banking_readwrite;
```

Exercise 3.6 — Inspect privileges

From `psql`:

SQL

```
\dn+
```

Using SQL:

SQL

```
SELECT grantee, privilege_type
FROM information_schema.role_table_grants
WHERE table_schema = 'banking'
      AND table_name = 'customers'
ORDER BY grantee, privilege_type;
```

Exercise 3.7 — Revoke a privilege

SQL

```
REVOKE DELETE
ON banking.customers
FROM banking_readwrite;
```

Verify:

SQL

```
\dp banking.customers
```

Lab 4 — User Administration

Objective

Create login users, assign group roles, test access, and manage account settings.

In PostgreSQL, a user is a role that has the `LOGIN` attribute.

Exercise 4.1 — Create users

While connected as the administrator:

SQL

```
CREATE ROLE audit_user
LOGIN
PASSWORD 'AuditUser@123';

CREATE ROLE application_user
LOGIN
PASSWORD 'ApplicationUser@123';
```

Exercise 4.2 — Assign group roles

SQL

```
GRANT banking_readonly TO audit_user;  
GRANT banking_readwrite TO application_user;
```

Check memberships:

SQL

```
\du audit_user  
\du application_user
```

Or:

SQL

```
SELECT member_role.rolname AS member,  
       group_role.rolname AS granted_role  
FROM pg_auth_members membership  
JOIN pg_roles group_role  
  ON group_role.oid = membership.roleid  
JOIN pg_roles member_role  
  ON member_role.oid = membership.member  
ORDER BY member, granted_role;
```

Exercise 4.3 — Test the read-only user

Exit:

SQL

```
\q
```

Connect as `audit_user`:

Bash

```
psql -h 127.0.0.1 -p 5432 \  
-U audit_user -d role_lab_db
```

Read operation should succeed:

SQL

```
SELECT * FROM banking.customers;
```

Write operation should fail:

SQL

```
INSERT INTO banking.customers (customer_name, city)
VALUES ('Test Customer', 'Mumbai');
```

Expected result:

```
ERROR: permission denied for table customers
```

Exit:

SQL

```
\q
```

Exercise 4.4 — Test the read-write user

Bash

```
psql -h 127.0.0.1 -p 5432 \
-U application_user -d role_lab_db
```

SQL

```
SELECT * FROM banking.customers;

INSERT INTO banking.customers (customer_name, city)
VALUES ('Kavitha', 'Hyderabad');

UPDATE banking.customers
SET city = 'Pune'
WHERE customer_name = 'Kavitha';
```

Because `DELETE` was revoked in Lab 3, this should fail:

SQL

```
DELETE FROM banking.customers
WHERE customer_name = 'Kavitha';
```

Exercise 4.5 — Change a password

Reconnect as `postgres`:

Bash

```
sudo -iu postgres psql
```

SQL

```
ALTER ROLE application_user  
PASSWORD 'NewApplicationUser@123';
```

Exercise 4.6 — Set password expiry

SQL

```
ALTER ROLE application_user  
VALID UNTIL '2026-12-31 23:59:59+00';
```

Inspect the expiry date:

SQL

```
SELECT rolname, rolvaliduntil  
FROM pg_authid  
WHERE rolname = 'application_user';
```

`pg_authid` is restricted to sufficiently privileged administrators.

Exercise 4.7 — Disable and enable login

Disable:

SQL

```
ALTER ROLE application_user NOLOGIN;
```

Verify:

SQL

```
SELECT rolname, rolcanlogin  
FROM pg_roles  
WHERE rolname = 'application_user';
```

Enable again:

SQL

```
ALTER ROLE application_user LOGIN;
```

Exercise 4.8 — Remove role membership

SQL

```
REVOKE banking_readwrite FROM application_user;
```

Confirm:

SQL

```
\du application_user
```

Restore it for subsequent testing:

SQL

```
GRANT banking_readwrite TO application_user;
```

Lab 5 — Tablespace Management

Objective

Create a tablespace and place a database, table, and index in the designated storage location.

Exercise 5.1 — Create the operating-system directory

Exit `psql`, then run:

Bash

```
sudo install -d \  
-o postgres \  
-g postgres \  
-m 700 \  
/data/pg_tablespace/fast_storage
```

Verify:

Bash

```
ls -ld /data/pg_tablespace/fast_storage
```

Expected owner:

```
postgres postgres
```

The tablespace directory must be:

- Empty
- Owned by the PostgreSQL operating-system user
- Outside the main PostgreSQL data directory
- Located on storage available whenever PostgreSQL is running

Exercise 5.2 — Create the tablespace

Bash

```
sudo -iu postgres psql
```

SQL

```
CREATE TABLESPACE fast_storage
OWNER postgres
LOCATION '/data/pg_tablespaces/fast_storage';
```

List tablespaces:

SQL

```
\db
```

Check their physical locations:

SQL

```
SELECT spcname,
       pg_tablespace_location(oid) AS location
FROM pg_tablespace;
```

Exercise 5.3 — Create a database in the tablespace

SQL

```
CREATE DATABASE analytics_db
TABLESPACE fast_storage;
```

Verify:

SQL

```
SELECT d.datname,  
       t.spcname AS default_tablespace  
FROM pg_database d  
JOIN pg_tablespace t  
     ON t.oid = d.dattablespace  
WHERE d.datname = 'analytics_db';
```

Exercise 5.4 — Create a table in the tablespace

SQL

```
\c analytics_db
```

SQL

```
CREATE TABLE transactions (  
    transaction_id BIGINT GENERATED ALWAYS AS IDENTITY,  
    account_number VARCHAR(20) NOT NULL,  
    amount NUMERIC(12,2) NOT NULL,  
    transaction_time TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP  
) TABLESPACE fast_storage;
```

Insert sample data:

SQL

```
INSERT INTO transactions (account_number, amount)  
VALUES  
    ('ACC1001', 1500),  
    ('ACC1002', 2750),  
    ('ACC1003', 3250);
```

Exercise 5.5 — Create an index in the tablespace

SQL

```
CREATE INDEX idx_transactions_account  
ON transactions(account_number)  
TABLESPACE fast_storage;
```

Exercise 5.6 — Verify object locations

SQL

```
SELECT c.relname AS object_name,  
       c.relkind AS object_type,  
       COALESCE(
```

```

        t.spcname,
        current_setting('default_tablespace'),
        'database default'
    ) AS tablespace
FROM pg_class c
LEFT JOIN pg_tablespace t
    ON t.oid = c.reltablespace
WHERE c.relname IN (
    'transactions',
    'idx_transactions_account'
);

```

A simpler `psql` check:

SQL

```
\d+ transactions
```

Exercise 5.7 — Move an existing table

Create another tablespace directory:

SQL

```
\q
```

Bash

```

sudo install -d \
-o postgres \
-g postgres \
-m 700 \
/data/pg_tablespace/archive_storage

```

Create the tablespace:

Bash

```
sudo -iu postgres psql -d analytics_db
```

SQL

```

CREATE TABLESPACE archive_storage
OWNER postgres
LOCATION '/data/pg_tablespace/archive_storage';

ALTER TABLE transactions
SET TABLESPACE archive_storage;

```

Move its index separately:

SQL

```
ALTER INDEX idx_transactions_account  
SET TABLESPACE archive_storage;
```

Verify:

SQL

```
\d+ transactions
```

Cleanup

Connect to another database before deleting `analytics_db`:

SQL

```
\c postgres
```

Terminate its remaining sessions if required:

SQL

```
SELECT pg_terminate_backend(pid)  
FROM pg_stat_activity  
WHERE datname = 'analytics_db'  
AND pid <> pg_backend_pid();
```

Remove the database:

SQL

```
DROP DATABASE analytics_db;
```

Remove the now-empty tablespaces:

SQL

```
DROP TABLESPACE fast_storage;  
DROP TABLESPACE archive_storage;
```

Exit and remove the empty operating-system directories:

SQL

\q

Bash

```
sudo rmdir /data/pg_tablespace/fast_storage
sudo rmdir /data/pg_tablespace/archive_storage
```

Final Cleanup for Roles Lab

Bash

```
sudo -iu postgres psql
```

SQL

```
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = 'role_lab_db'
    AND pid <> pg_backend_pid();

DROP DATABASE role_lab_db;

DROP ROLE audit_user;
DROP ROLE application_user;
DROP ROLE banking_readonly;
DROP ROLE banking_readwrite;
```

Expected Learning Outcomes

After completing these labs, participants can:

- Create and administer PostgreSQL databases.
- Create and operate multiple PostgreSQL clusters.
- Distinguish a database from a PostgreSQL cluster.
- Design group roles using least-privilege access.
- Create login users and manage their role memberships.
- Test read-only and read-write permissions.
- Manage passwords, expiry dates, and login status.
- Create and manage tablespaces.
- Place databases, tables, and indexes on selected storage.

